

Extreme Programming

Lecture 06

1

Case Study



Narrative: a team working on a fantasy basketball website

Justin – a developer

Danielle – another developer

Bridget – their project manager

Act I: Going into Overtime

Agile Software Development

2

2

Extreme Programming

- Extreme Programming (XP) is an agile methodology.
- It's made up of **practices, values, and principles**.
- Practices are easy to learn, and they're highly impactful.
- But change only happens if the people on the team approach it with the right mindset.

Agile Software Development

3

3

Primary Practices of XP

- There are 13 primary practices of XP.
- Many of the XP practices are specific to programming.
- They are aimed at addressing the most common problems that cause teams to build lousy code.

Agile Software Development

4

4

Programming Practices

- Test-first programming helps team more than just making sure the correct execution of code.
- It also helps to prevent some of the most common and most serious problems that can make a codebase difficult to maintain.
- It helps identifying the issues related to the dependencies with in the codebase.

Agile Software Development

9

9

Programming Practices

- That is by using automated tests that helps prevent dependencies from going undetected by immediately failing.
- Tests help the programmer spot the problem before it's embedded in the codebase and too difficult to remove.
- Unit tests also help programmers write code that is easier to reuse.

Agile Software Development

10

10

Programming Practices

Pair Programming Practice

- Two developers sit together at a single workstation when writing code.
- In most cases, one programmer types, while the other watches.
- Teams working in pairs inject far fewer bugs.
- Pairing can also help reduce fatigue.

Agile Software Development

11

11

Programming Practices

- An effective pair programming team will discuss ideas and approaches, constantly brainstorm, and, as a result, be more innovative.
- They will also hold each other accountable for good practices.
- Leads to much better code than they would have if working separately.

Agile Software Development

12

12

Integration Practices

- There are two primary XP practices under the Integration practices.
 - 10-minute build
 - Continuous integration



Agile Software Development

13

13

Integration Practices

10-minute Build practice

- Team creates an automated build for the entire codebase that runs in under 10 minutes.
- This build includes
 - automatically running all of the unit tests
 - generating a report of which tests passed and which failed.

Agile Software Development

14

14

Integration Practices

- Running the build often is very valuable to the team, because it makes problems immediately obvious.
- **“Is our code working so far?”**
- Because it's short enough that the team will run it often, everyone on the team has a continuously updated view of the quality of the code.

Agile Software Development

15

15

Integration Practices

Continuous Integration Practice

- In a team, everyone needs to work with the same files all the time.
- Team members can't work on the same physical copies of the files at the same time.
- They might constantly overwrite one another's changes.

Agile Software Development

16

16

Integration Practices

- So, team will use a version control system that has a central master set of files.
- Individual developers will **check-out**, or copy into a private copy of the entire codebase (often called a **sandbox**).
- Developer will work on the copy of the code in her sandbox, and periodically **check-it** back into the repository.

Agile Software Development

17

17

Integration Practices

- During the check-in of changes, developer might face conflicts with another members' changes.
- These will show up immediately when she tries to build her local copy of the code, and it no longer compiles.
- Sometimes software no longer behaves correctly because of conflicting code that was checked in since last update of sandbox.

Agile Software Development

18

18

Integration Practices

- In continuous integration, team will constantly run the build and monitor for compiler errors or unit test failures.
- Team set up a build server that periodically
 - checks out the latest code from the repository.
 - runs the automated build.
 - notifies the team if there are any errors.

Agile Software Development

19

19

Integration Practices

- This is only one part of continuous integration.
- Also includes that each team member constantly keeps his or her own copy of the codebase up to date.
- Team members periodically integrate the latest code from the repository back into their sandboxes.

Agile Software Development

20

20

Planning Practices

- XP teams use iterative development to manage their projects.
- Like Scrum, the XP planning practices are based on a larger cycle for long-term planning, broken into shorter iterations.



Agile Software Development

21

21

Planning Practices

Weekly Cycle practice

- XP team uses one-week iterations, and it works closely with the stories practice.
- Each cycle starts with a planning meeting.
 - Team members review the progress.
 - Work with the customers to pick the stories for the iteration.
 - Break the stories into tasks that are estimated and assigned to developers.

Agile Software Development

22

22

Planning Practices

- After planning, team spends the first part of the iteration writing automated tests for the stories and tasks.
- Rest of the iteration is for writing code to make the tests pass.
- Teams will stack up all of the tasks for the iteration.
- Each developer take the next task in the stack when his current task is done.

Agile Software Development

23

23

Planning Practices

Quarterly Cycle practice

- This is to do long-term planning.
- Once a quarter, the team meets to take a look at the big picture.
- Teams will talk about themes, or larger ideas in the real world that they can use to tie their project's stories together.

Agile Software Development

24

24

Planning Practices

- Also talks about internal and external problems that they're experiencing, nagging bugs, and repairs that haven't been addressed yet.
- Reflect on the progress they've made so far, how well they're meeting their users' needs, and how the project is going overall.

Agile Software Development

25

25

Planning Practices

Slack Practice

- Team adds minor, lower-priority stories to each weekly cycle.
- These stories are broken down into tasks during the planning meeting, but those tasks are left for the end of the iteration.
- If team runs into unexpected problems, these slack stories will be omitted from the release.

Agile Software Development

26

26

Team Practices

- XP isn't just about programming.
- It is about how teams work together.
- It includes two primary practices that help teams gel.
 - Sit together
 - Informative workspace



Agile Software Development

27

27

Team Practices

Sit Together practice

- Teams work best when they sit near each other, and have easy access to everyone else on the team.
- Although programming as an individual is often done in isolation, working as a developer on a programming team is a highly social activity.

Agile Software Development

28

28

Team Practices

- Team members constantly
 - consult each other about problems
 - ask each other for advice
 - alert each other to problems
- If the team sits together in an open workspace, this socialization is naturally encouraged.

Agile Software Development

29

29

Team Practices

Informative Workspace practice

- Team's working environment is set up to automatically communicate important project information.
- Large task board and burndown chart posted prominently on a wall that everyone can see.

Agile Software Development

30

30

Team Practices

- Every team member knows where the project stands, and that helps everyone make better decisions.
- Also, When team members have questions, run into problems, or raise issues, they have discussions.
- These will take place in a shared workspace, the people around them overhear and will absorb information.

Agile Software Development

31

31

Embracing Change with XP

- An XP team doesn't just treat changes as a necessary evil.
- They recognize that the only way they can build the best software for their users is to get feedback often and respond to that feedback quickly.
- They expect change; it's on their calendar.

Agile Software Development

32

32

Embracing Change with XP

- Embracing change has two aspects:
 - on the scope of the project
 - on the codebase itself
- Impact of changes on the code can be serious.
- Primary practices of XP helps teams to embrace changes to the scope, functionality, and behavior of the software.

Agile Software Development

33

33

Embracing Change with XP

Planning practices

- XP teams deliver working software at the end of every iteration.
- Feedback is also available at the end.
- Every team member knows they're actively asking for that changes as feedback.
- Much easier to deal with changes.

Agile Software Development

34

34

Embracing Change with XP

Team practices

- When whole team is readily available to help it is lot easier to deal with the problems.
- Everyone is thinking about the same problems.
- Gives team members the ability to take control of those changes when they happen.

Agile Software Development

35

35

Embracing Change with XP

Integration practices

- Usually changes, ripple out through the entire codebase.
- This leads to bugs in the codebase.
- Continuous integration helps by finding out the problem very quickly, and fix it early.

Agile Software Development

36

36

Embracing Change with XP

Programming practices

- During the integration of the change to the codebase, test related to the changed codebase will fail.
- Team can work together to fix the problem in both parts of the code.

Agile Software Development

37

37

Case Study: Continued



Narrative: a team working on a fantasy basketball website

Justin – a developer

Danielle – another developer

Bridget – their project manager

Act II: The Game Plan Changed, but We're Still Losing

Agile Software Development

38

38

Values of XP

- XP has five values.
- These values helps team to truly adopt XP and avoid better-than-not-doing-it results.
- XP values help get the team into a mindset where great practices seem like the obvious way to build great software.

Agile Software Development

39

39

Values of XP

1. Communication

- Each team member is aware of the work everyone else is doing.

2. Simplicity

- Developers focus on writing the most simple and direct solutions possible.

3. Feedback

- Constant tests and feedback loops keep the quality of the product in check.

Agile Software Development

40

40

Values of XP

4. Courage

- Each team member is focused on making the best choices for the project, even if it means having to discard failing solutions or approach things differently.

5. Respect

- Every team member is important and valuable to the project.

Agile Software Development

41

41

Understanding XP Values

- When a team starts their XP adoption, everyone is optimistic.
- Team expects to get their problem fixed with the adoption of XP practices.
- But in real life it does not happen as expected.



Agile Software Development

42

42

Understanding XP Values

Example: Pair Programming

- It feels very expensive having someone sit with you as you go through the thought process of writing code.
- Code review might be a option.



Agile Software Development

43

43

Understanding XP Values

- Almost every XP practice has a shortcut.
 - Pair programming -> Code review
 - Sit together -> Daily meetings
 - No continuously integration -> Automated server
- And in every case, it's better than not doing anything.
- So you'll get better-than-not-doing-it results.

Agile Software Development

44

44

Understanding XP Values

- XP practices help the team bring the values of XP into the project.
- Shortcuts change the mechanics of how the team does their work.
- But they don't change the intent or the mindset of the team.
- Teams new to XP, skip the values, because the practices seem more interesting.

Agile Software Development

45

45

Understanding XP Values

- Practices are concrete.
- It is easy to understand how a individual practice impact the project.
- Practices are also isolated.
- It is easy to add an individual practice to the project without having to change the way that team think about the project.

Agile Software Development

46

46

Understanding XP Values

- Values are harder for teams to get their heads around.
- They're more abstract.
- Values affect the entire project and the way everyone thinks about it.
- Adding add practices to project without understanding the values gives better-than-not-doing-it results.

Agile Software Development

47

47

Understanding XP Values

- When the XP values aren't truly internalized by each person on the team, they're likely to get better-than-not-doing-it results
- A good XP team has the right mindset when they really get the values.
- And when they do, the practices start to make sense.

Agile Software Development

48

48

Understanding XP Values

- Imagine a team new to XP starts doing continuous integration.
- How will this help the team learn about XP values?
 - Communication
 - Respect
 - Feedback
 - Courage

Agile Software Development

49

49

Case Study: Continued



Narrative: a team working on a fantasy basketball website

Justin – a developer

Danielle – another developer

Bridget – their project manager

Act III: The Momentum Shifts

Agile Software Development

50

50

Principles of XP

- There's a gap between the values of XP and its practices.
- For people new to XP, it's sometimes hard to apply values to specific practices.
- XP has a set of principles that help guide in applying the practices to projects in the real world.

Agile Software Development

51

51

Principles of XP

- Values of XP are broad.
- So the principles help fill in a lot of the details about how people on an XP team think about their problems.
- This makes them very important in helping you figure out if your team has the right mindset for XP.

Agile Software Development

52

52

Principles of XP

- Humanity
 - Remember that software is built by people, and there's a balance between the needs of each team member and the project's needs.
- Economics
 - Somebody is always paying for software project and everybody has to keep the budget in mind.

Agile Software Development

53

53

Principles of XP

- Mutual benefit
 - Search for practices that benefit the individual, the team, and the customer together.
- Self similarity
 - The pattern of a monthly cycle is the same as a weekly cycle and the same as a daily cycle.
- Improvement
 - Do your best today, and stay aware of what you need to do to get better tomorrow.

Agile Software Development

54

54

Principles of XP

- Diversity
 - Lots of different opinions and perspectives work together to come up with better solutions.
- Reflection
 - Good teams stay aware of what's working and what isn't in their software process.

Agile Software Development

55

55

Principles of XP

- Flow
 - Constant delivery means a continuous flow of development work, not distinct phases.
- Quality
 - You can't deliver faster by accepting a lower quality product.

Agile Software Development

56

56

Principles of XP

- Accepted responsibility
 - If someone is responsible for something, they need to have the authority to get it done.
- Baby steps
 - Take small steps in the right direction rather than making wholesale changes when adopting new practices.

Agile Software Development

57

57

Understanding Principles of XP

- These principles are useful to help you understand some of the specific practices that XP uses.
- And when you start exploring the practices, they help you understand the principles.

Agile Software Development

58

58

Understanding Principles of XP

- Principles are about self reflection.
- They force you to take a step back and think about how you and your team do your jobs.
- When you take the time to understand the values and principles, you start to learn where your team has problems.

Agile Software Development

59

59